SETUP GUIDE · WORKS ON ANY GIT REPOSITORY

Adopting the Harness

Install it in stages, smallest first. Each stage is useful on its own, and nothing later depends on you having finished everything earlier.

◆ START HERE IF YOU ONLY HAVE AN AFTERNOON

Do **Stage 0** and **Stage 1**. Stage 0 is a settings change with no code and it is the difference between having controls and having suggestions. Stage 1 is one small file that changes how every check you ever write behaves. Everything after that is optional and incremental.

REQUIRES
Git · Python 3 · CI

LANGUAGE
Any — gates are language-agnostic

COMPANION
Pattern Handbook (the why)

00 The stages, and what each buys you

STAGE	WHAT YOU ADD	WHAT IT BUYS
0	Branch protection	**Everything else becomes binding.** Without it every control below only advises.
1	The exit-code contract	A check that could not run stops reading as a pass.
2	Your first gate + its fault suite	One rule enforced, and proof the enforcement works.
3	The decision register	Blocked questions get recorded instead of guessed.
4	Pre-commit hook + CI wiring	The gates actually run, in both places.
5	Secret scanning	A credential is caught before it becomes permanent.
6	Mutation testing	A measure of verification rather than execution.
7	Trigger gates	Controls you will need later start failing when they start mattering.
8	Drift protection	Shared files across repositories stop rotting silently.

What is in the kit

```
tools/gates/
  harness.py                       the exit-code contract (Stage 1)
  check_decisions.py               the decision register gate (Stage 3)
  check_vendored_drift.py          shared-file drift (Stage 8)
  check_mutation_applicability.py  a worked trigger gate (Stage 7)
  check_gates_test.py              fault injection for all of the above (Stage 2)
  expected-gates.txt               the register of gates that must exist
tools/qa/
  run-qa.sh                        runs the floor, writes a durable record
docs/
  DECISIONS.md                     an empty register with its schema
.githooks/pre-commit               discovers and runs every gate
.github/workflows/                 gates + secret scanning
.gitleaks.toml                     secret-scanning config
examples/vendored.json             template for the drift manifest
```

No third-party Python packages, deliberately. A gate that needs `pip install` to work has acquired a way to fail that has nothing to do with the property it checks — and that failure looks identical to the property being broken.

01 Stage 0 — branch protection first

> ✗ **DO THIS BEFORE WRITING A SINGLE GATE**
>
> Every control in this kit is advisory until your host enforces it. Skip this stage and you will build a careful harness that nothing obeys — which is measurably what happens. In the implementation this kit came from, before protection existed: **134 of 134 changes merged by their own author, zero approvals ever recorded, median 39 seconds from open to merge.** The gates were good. Nothing ran them.

Configure on your default branch

SETTING	WHY
Require status checks to pass	Name each gate job explicitly. A check that is not required is a suggestion.
Require at least one approving review	Two if you can. With agents, prefer reviewers of a *different model family* from the author.
Dismiss stale approvals on push	Otherwise an approval covers code nobody approved.
Do not allow bypass	Including for administrators. An exception used once becomes the route.
Require a linear history *(optional)*	Makes "what actually landed" answerable without archaeology.

Verify it rather than assuming it

```
# GitHub: confirm protection exists and lists your checks
gh api repos/<org>/<repo>/branches/main/protection

# "Branch not protected" means Stage 0 is not done, whatever the settings page shows.
```

Add a code owners file while you are here

It routes review by path. One line per area of risk. **It does nothing until protection requires code-owner review** — worth knowing, because a code owners file that nobody is required to satisfy reads like a control and is not one. If you are a single maintainer, write yourself and say in a comment that it is a placeholder; inventing team handles that do not exist produces silently broken review requests.

02 Stage 1 — the exit-code contract

One file, no dependencies, and it changes the shape of every check you write afterwards. Copy `tools/gates/harness.py` as-is.

EXIT	MEANS	USE IT WHEN
0	Verified	The check ran and the property holds.
1	Violated	The check ran and the property is broken.
2	Could not run	A tool was missing, a credential unset, a service down. **Not a pass.**
3	Incomplete	It ran, but part of what it should cover was unreachable — so that part is *unproven*. **Not a pass.**

Then audit what you already have

Look for these three shapes in existing scripts and CI, and fix them:

FOUND	CHANGE TO
`some-check || true`	Let it fail. If it genuinely may fail, mark it advisory *deliberately* and say why.
`if ! command -v tool; then exit 0; fi`	`exit 2` — a missing tool means no verdict, not a good one.
`A check whose partial coverage returns 0`	`exit 3`, and name what went unchecked.

> ◆ **ENFORCE IT IN REVIEW, NOT JUST IN CODE**
>
> The contract is only worth having if callers respect it. The question to ask of any new check is **"what does this return when it cannot do its job?"** If the answer is 0, the check will eventually go quiet and nobody will notice.

03 Stage 2 — your first gate, and its fault suite

Pick the rule you would most hate to see broken. Write the smallest check that catches it. Then — before trusting it — write the test that proves it can fail.

The gate

```
#!/usr/bin/env python3
"""One-line statement of the property. Then WHY it matters — the comment that
saves the next person from re-deriving your reasoning."""
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
from harness import VERIFIED, VIOLATED, CANNOT_RUN, repo_root, report

ROOT = repo_root()

def main() -> int:
    if not (ROOT / "some/required/input").exists():
        return report("my gate", CANNOT_RUN,
                      violations=["the input this checks is absent"])
    problems = [...]                      # your actual check
    if problems:
        return report("my gate", VIOLATED, violations=problems)
    return report("my gate", VERIFIED, verified=["what was checked, and how much"])

if __name__ == "__main__":
    sys.exit(main())
```

The fault suite — the part people skip

> ◆ **WHY YOU CANNOT SKIP IT**
>
> **A gate that always passes is indistinguishable from a codebase that is always correct.** The output is identical. A typo in a glob or a swallowed exception produces a check that is green forever, and nothing about green invites suspicion.

Four requirements, each earned from a real failure:

REQUIREMENT	WHY
Make the violation real	In a temporary directory, create the actual bad condition. A mocked violation proves the mock.
Assert the exact exit code	1 and 3 are different claims. "Any failure" passes while your gate conflates them.
Include negative cases	A gate that fires on everything gets disabled within a week.
Run it even when the gate is red	That is exactly when "is the gate still sound?" matters.

`tools/gates/check_gates_test.py` in this kit is a working example covering 22 cases across three gates. Run it:

```
python3 tools/gates/check_gates_test.py
```

When first run against this kit's own gates it found **seven real defects** — every one a silent pass or a mis-report, and none visible by reading. Two are worth knowing before you write your own: a source scan that allow-listed four directory

names silently ignored code anywhere else, and **fixing that created the opposite bug**, where the harness counted its own scripts as product source. Fixing a silent pass in one direction opened one in the other — which is the real argument for the suite.

04 Stage 3 — the decision register

Copy `docs/DECISIONS.md` and `tools/gates/check_decisions.py`. The file starts empty, and an empty register passes — the gate says it validated *zero rows* rather than reporting a pass as though it had checked something.

Tailor these three constants

CONSTANT	CHANGE TO
`ID_PREFIX`	Your convention. `DEC`, `ADR`, `OPEN` — anything, as long as it is stable, because these ids get cited.
`REQUIRED_FIELDS_OPEN`	Add fields your team needs. **Do not remove the issue link or the code-difference field** — they are what make the register work.
`CITATION_GLOBS`	Where you write documents that might cite a decision.

> ◆ **TAILORING VERSUS WEAKENING**
>
> Adding a field is tailoring. Removing a check because a row fails it is **deleting the finding**. The distinction is worth stating out loud in your contributing guide, because the two look identical in a diff.

Writing a row people can act on

The two fields that decide whether the register is useful:

- **Issue link.** This is the delivery mechanism — the tracker's assignment notification is what actually reaches a human. A row without one has told nobody anything.
- **What differs in the code.** Name what would be *physically different* under each option, with a real path or symbol. If you cannot, this is a task, not a decision, and the gate rejects it. That test is what stops the register filling with to-do items.

Then use it, which is the hard part

The register only works if "stop and record" beats "pick the sensible option". Make it explicit in your agent instructions and your contributing guide: **if the requirement does not settle it, record a decision and route around it.** An agent will always produce a confident answer, so this has to be a stated rule rather than an assumed instinct.

05 Stages 4 and 5 — wiring, and secret scanning

Stage 4 — make the gates actually run

COMMAND	WHAT IT DOES
`git config core.hooksPath .githooks`	Opt in to the pre-commit hook. It **discovers** gates — every `check_*.py` that is not a test — so a new gate joins the moment it lands.
`bash tools/qa/run-qa.sh`	Runs the whole floor and writes a durable record to `tools/qa/evidence/latest.md`. **Commit that file** — evidence must outlive the terminal.

Copy `.github/workflows/gates.yml` for CI. Note the `if: ${{ !cancelled() }}` on every step after the first: without it, one failing check silently skips the rest, and a reader cannot tell a check that passed from one that never ran.

> ◆ **WHY THERE IS A REGISTER OF EXPECTED GATES**
>
> Discovery has one blind spot: it cannot tell *"this gate does not exist"* from *"this gate was deleted"*. A run that reports green because it found nothing to run is the same defect as a report recording NOT RUN as measured. `expected-gates.txt` closes it — a name listed there and missing from the checkout blocks. Land a gate and its row in the same change.

The hook is a convenience, not a control. `--no-verify` skips it and every developer has that flag. It catches the honest mistake where it is cheapest to fix, and stops nothing determined. That is Stage 0's job.

Stage 5 — secret scanning

Copy `.gitleaks.toml` and `.github/workflows/secret-scan.yml`. Three details that matter more than the tool choice:

DETAIL	WHY
Scan full history	A shallow clone scans only the tip and reports "no leaks" for a repo it barely looked at.
Always redact	Without it, a detected secret is printed into a CI log — turning the detector into a *second* disclosure channel. File, line and rule are still reported, which is all you need to fix it.
Never allowlist by value	Suppressing a specific string is how a real credential gets permanently whitelisted by someone in a hurry, invisibly. Allowlist by *path*, with a comment saying why.

And do **not** allowlist a file that legitimately holds live credentials. Git-ignore it instead. If someone force-adds it, the scan must fire — that is the entire point.

Verify it works by proving it fires: commit a randomly generated fake credential to a tracked file on a throwaway branch, confirm a non-zero exit, then discard the branch. An unverified scanner is a scanner you are trusting on faith.

06 Stages 6 to 8 — measurement, triggers, drift

Stage 6 — mutation testing, replacing your coverage target

Pick your language's tool, configure it for **one** package, and set a floor. Then **delete your line-coverage gate** — keeping both means the weaker one still shapes behaviour, and coverage actively rewards assertion-free tests.

RULE	WHY
A surviving mutant is a missing assertion	Add the assertion. Do not lower the floor.
Quote the denominator	"99%" can be nearly meaningless. Say how many mutants were actually evaluated, and how many were excluded and why.
Refuse to emit a score from an empty run	If everything was excluded, there is no score. A number derived from nothing is worse than no number, because it looks authoritative.

Stage 7 — trigger gates for controls you do not need yet

Use `check_mutation_applicability.py` as the worked example. The shape: distinguish **not applicable** (nothing to measure — passes, *and names its trigger*) from **unobtainable** (something to measure and no way to measure it — fails).

Test it as a state machine, not a single run — add the thing that should trip it, confirm it goes red, remove it, confirm it goes green. A trigger you have not watched fire is not a trigger.

Good candidates: accessibility checks (trigger: first user-facing surface), load tests (first served traffic), migration safety (first schema), licence scanning (first third-party dependency). Each is normally a checklist item nobody actions.

Stage 8 — drift protection, when a second repository appears

Copy `examples/vendored.json` to `tools/gates/vendored.json` and fill it in. Until that file exists the gate reports *not applicable* and passes, so a single-repo project is never nagged.

RULE	WHY
Keep copies byte-identical	A file adapted locally is a file nobody can diff. Put local context in a document, never in the file.
Change upstream, then re-copy	Never keep a local edit and re-hash it — that is how a fork becomes permanent.
Say what a pass means	*Unchanged since copied*, not *up to date*. Detecting upstream moving ahead needs network access and a credential.
Record the copying as a decision	It is a workaround for not having a published package. Name it in the register or you have chosen by default.

07 Verifying your installation

Run these in order. Every one should behave as described — and if one does not, that is a finding about your setup rather than something to work around.

COMMAND	EXPECT	MEANING
`python3 tools/gates/check_gates_test.py`	**0**	Every gate fires on a real violation and stays quiet otherwise. **Run this first** — it validates the validators.
`python3 tools/gates/check_decisions.py`	**0**	On an empty register, says it validated zero rows.
`python3 tools/gates/check_mutation_applicability.py`	**0** or **1**	0 if you have no source matching the globs (check the globs are right — a scan matching nothing looks like a clean project); 1 if you have source and no mutation config, which is the correct verdict until Stage 6.
`python3 tools/gates/check_vendored_drift.py`	**0**	"Not applicable" until you create the manifest.
`bash tools/qa/run-qa.sh`	**0** or **1**	Writes the evidence record either way. Read the table it produces.
`gh api repos/<org>/<repo>/branches/main/protection`	JSON	**"Branch not protected" means Stage 0 is not done**, and everything above is advisory.

Then deliberately break something

This is the step that tells you the installation is real. On a throwaway branch:

```
# 1. Add a decision row with no issue link      → check_decisions.py must exit 1
# 2. Add a source file with no mutation config   → applicability gate must exit 1
# 3. Commit a randomly generated fake credential → secret scan must exit non-zero
# 4. Delete a gate named in expected-gates.txt   → the hook must refuse the commit

git checkout -b throwaway/verify-harness
# ... make one of the above changes, observe the failure, then:
git checkout - && git branch -D throwaway/verify-harness
```

If any of those *passes*, you have a gate that cannot fail — which is the one outcome worse than having no gate, because it will be believed.

Common installation problems

SYMPTOM	CAUSE
A gate passes on an obviously broken repo	Its globs match nothing. A scan that finds no files is indistinguishable from a clean project — check the file count it reports.
The decision gate flags your own documentation	Prose that *discusses* an id must write it unnumbered, because a real id in a comment is a citation the scan will rightly demand a row for.
CI green, but a check never appears in the log	An earlier step failed and skipped it. Add the failure-surviving condition.
A gate works locally, fails in CI as "could not run"	The job does not install a tool the gate shells out to. Install it in that job.
Everything passes and you do not believe it	Correct instinct. Go and break something — the paragraph above.

Adopting the Harness — Setup Guide. Companion to **The Agent Development Harness — Pattern Handbook**, which explains the reasoning behind each control. The reference implementation's own gates were verified by the procedure in §07, and doing so found seven real defects in them — including one created by the fix for another. That is the strongest argument this guide can offer for actually running it.